

Part 7: Diameter of a Binary Tree

The classic bottom-up problem

What diameter means

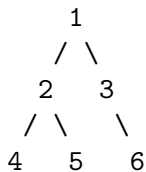
The **diameter** of a binary tree is the length of the longest path between any two nodes in the tree.

Two things people miss on first reading:

1. The path does **not** have to pass through the root.
2. The path does **not** have to start or end at a leaf, although in practice the longest one always does.

LeetCode 543 measures the length in **edges**. So a single node has diameter 0, and two nodes joined by one edge have diameter 1. We will use edges throughout. If a problem asks for the number of nodes on the path instead, the answer is simply edges + 1.

On our tree:

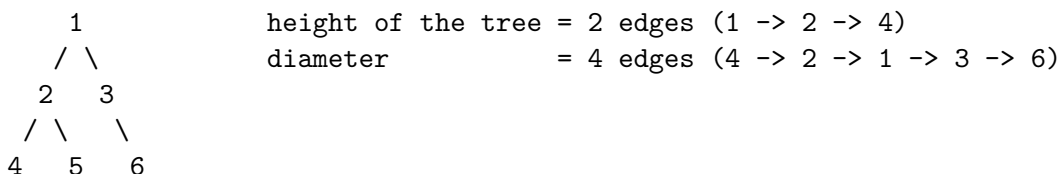


The longest path is 4 to 2 to 1 to 3 to 6, which uses 4 edges. So the diameter is 4. Another candidate is 4 to 2 to 5, which uses only 2 edges.

Height is not diameter

These get confused constantly.

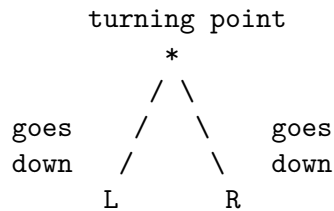
- **Height** measures a path that goes straight down from one node. It never turns.
- **Diameter** measures a path that may go up from one node and then down into a different branch. It turns exactly once.



The diameter is at least the height and can be almost twice as much.

The key observation

Every path in a tree has a single **highest** node, the point where it turns. Call it the turning point.



Once you fix the turning point, the best possible path through it is forced: go as deep as possible into the left subtree, and as deep as possible into the right subtree.

So, if $h(x)$ is the height of the subtree rooted at x measured in **nodes** (so $h(\text{NULL}) = 0$), then

best path turning at node = $h(\text{node.left}) + h(\text{node.right})$ [in edges]

Why does that come out in edges? Going down $h(\text{left})$ nodes from node uses exactly $h(\text{left})$ edges, because the first edge takes you from node into the left subtree. Same on the right. Add them.

And then:

diameter of tree = maximum over all nodes of ($h(\text{left}) + h(\text{right})$)

Every path is counted exactly once, at its own turning point. Nothing is missed and nothing is double counted.

Brute force, and why it is slow

Translate the formula directly:

```
int height(TreeNode* root) {
    if (root == NULL) return 0;
    return 1 + max(height(root->left), height(root->right));
}

int diameterBrute(TreeNode* root){
    if (root == NULL) return 0;

    int through = height(root->left) + height(root->right); // path turning here
    int left    = diameterBrute(root->left);                // best path entirely on the left
    int right   = diameterBrute(root->right);               // best path entirely on the right

    return max(through, max(left, right));
}
```

This is correct. It is also wasteful. At every node we call `height`, which walks the entire subtree below that node. We computed those heights already inside the outer recursion and threw them away.

Cost on a skewed tree:

node 1: height walk over N nodes

node 2: height walk over N-1 nodes

...

total = $O(N^2)$

For $N = 10^5$ that is 10^{10} operations. Time limit exceeded.

The $O(N)$ solution

Use Part 6. Do one postorder pass. Each call returns **only the height**, and along the way it updates a shared best answer.

```
class Solution {
    int best = 0;

    int height(TreeNode* root) {
        if (root == NULL) return 0;

        int L = height(root->left);
        int R = height(root->right);

        best = max(best, L + R);           // the path that turns HERE
        return 1 + max(L, R);             // what my parent can use
    }

public:
    int diameterOfBinaryTree(TreeNode* root) {
        best = 0;                         // reset, LeetCode reuses the object
        height(root);
        return best;
    }
};
```

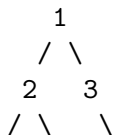
Six lines of logic. Read the two important ones again:

```
best = max(best, L + R);    // answer through this node, uses BOTH children
return 1 + max(L, R);      // value for the parent, uses ONE child
```

A path cannot go down the left branch, come back up, and then go down the right branch and still be a straight downward path for the parent. So the parent may only receive $1 + \max(L, R)$. The $L + R$ combination is real, but it is a completed path, so it is recorded and not passed on.

If you understand this distinction, you will get maximum path sum (LeetCode 124) right the first time, because it is the same shape with sums instead of counts.

Dry run



4 5 6

```

height(4): L=0 R=0   best = max(0, 0+0) = 0   return 1
height(5): L=0 R=0   best = max(0, 0)   = 0   return 1
height(2): L=1 R=1   best = max(0, 1+1) = 2   return 2
height(6): L=0 R=0   best = max(2, 0)   = 2   return 1
height(3): L=0 R=1   best = max(2, 0+1) = 2   return 2
height(1): L=2 R=2   best = max(2, 2+2) = 4   return 3

```

answer = 4

Trace the winning line. At node 1, $L = 2$ means the left subtree can reach 2 levels down (nodes 2 then 4), and $R = 2$ means the right subtree can reach 2 levels down (nodes 3 then 6). So the path is 4, 2, 1, 3, 6 with 4 edges. Exactly what we predicted from the picture.

Note also that best was 2 at one point, from the path 4 to 2 to 5, and later improved. That is the algorithm considering every turning point in turn.

Complexity

Version	Time	Space
Brute force	$O(N^2)$ worst case, $O(N \log N)$ if balanced	$O(H)$
Bottom-up	$O(N)$	$O(H)$

The bottom-up version enters each node once and does $O(1)$ work there. The space is the recursion stack, so $O(H)$.

Variations you should be able to handle

- **Diameter in nodes instead of edges:** return $\text{best} + 1$, or track $L + R + 1$ inside.
- **Weighted edges:** replace $1 + \max(L, R)$ with $w + \max(L, R)$ and $L + R$ with $w_l + w_r$ style bookkeeping.
- **Maximum path sum (LeetCode 124):** same skeleton. Return $\text{node} \rightarrow \text{val} + \max(0, \max(L, R))$ upward, record $\text{node} \rightarrow \text{val} + \max(0, L) + \max(0, R)$. The $\max(0, \dots)$ is there to drop negative branches. See Part 9.
- **Longest univalue path (LeetCode 687):** same skeleton, but a child only contributes if its value equals the parent's value.

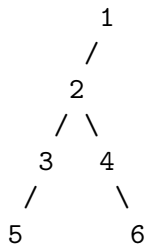
Every one of those is the same two lines with a different arithmetic. That is why diameter is worth this much attention.

Common mistakes

- **Returning $L + R$ to the parent.** This is the big one. It produces answers that are too large on some trees and correct on others, so small tests may pass.
- **Recomputing height at every node.** Correct but $O(N^2)$.
- **Mixing edges and nodes.** Decide first. With $h(\text{NULL}) = 0$ you are counting nodes for the height and $L + R$ naturally comes out in edges.
- **Forgetting to reset best.** Fails on the second test case only, which is confusing to debug.
- **Assuming the diameter passes through the root.** Build a counterexample: a root with a tiny right subtree and a long, deep left subtree whose two branches are both long.
- **Assuming the answer equals $2 * \text{height}$.** Only true for perfectly balanced shapes.

Check yourself

1. Why does every path in a tree have exactly one turning point?
2. Why is $h(\text{left}) + h(\text{right})$ a count of edges when h counts nodes?
3. What exactly is being recomputed in the brute force version?
4. What would break if `height` returned $1 + L + R$?
5. On this tree, what is the diameter?



6. State in one sentence the difference between the value recorded at a node and the value returned from it.